

Reservoir: Implementation Plan

Certified Exact Prioritized Experience Replay - Benchmarks & Application Roadmap

1. Project Overview

Reservoir is a correctness-oriented research implementation of Prioritized Experience Replay (PER) for reinforcement learning. Most PER implementations use floating-point arithmetic throughout, which can introduce subtle numerical errors. Reservoir proves that PER can be implemented with exact integer arithmetic - with provably zero floats on any decision path (except one explicitly declared boundary).

The 4 Research Theses

#	Claim	Status
T1	PER can be implemented with zero floats on any decision path	ALIVE (168 tests)
T2	Float idioms produce decision-relevant divergences from exact reference	DEAD (falsified)
T3	Durable buffer is failure-atomic under SIGKILL	ALIVE (70/70)
T4	Independent checker verifies sampled batches; forgeries rejected	ALIVE (63/63)

T2 is the prominently-reported negative result: the preregistered search found zero decision-relevant divergences between exact and float sum-tree idioms across 135 workloads. Total-variation distance stayed below 2^{-40} in every cell.

2. Architecture & Key Concepts

The float64-once Boundary (rational.py)

The ONLY float operation allowed is p^α - computed once in IEEE 754 float64, then immediately frozen to an exact integer:

```
priority_int = int(Fraction(p ** alpha) * 2**52)
```

All downstream arithmetic (tree sums, sampling, IS weights) uses Python integers and `fractions.Fraction`. This is the declared semantic boundary.

The Sum-Tree (sumtree.py)

A binary heap where each internal node stores the SUM of its subtree's leaf priorities. Enables $O(\log N)$ proportional sampling: generate a random integer in $[0, \text{total})$, walk the tree to locate which leaf it lands in. A parallel MinTree tracks the minimum priority for IS weight normalization. All arithmetic is pure Python integers.

Keyed Deterministic Draw (draw.py)

Reservoir - Implementation Plan

No random, numpy.random, or torch.rand on any decision path. Samples use BLAKE2b-256 keyed by (seed, buffer_id, op_counter) to produce a deterministic uniform integer. Every sampling decision is reproducible and auditable.

Crash-Safe Durable Buffer (durable.py)

Write-Ahead Log (WAL) commit protocol:

- Write intent record to intent.json.tmp
- F_FULLFSYNC intent.json.tmp (Darwin) / fsync (Linux)
- Write segment files with data
- F_FULLFSYNC each segment
- rename(intent.json.tmp, intent.json) <-- atomic commit point
- F_FULLFSYNC parent directory

Attestation Chain (attest.py)

Every insert/update/sample appends a hash-chained MutationRecord or SampleAttestation JSON record (BLAKE2b-256). The independent verifier checker/verify.py imports NOTHING from src/reservoir and re-implements the sum-tree from scratch to verify every record.

Fast Production Buffer (fast_buffer.py)

The exact buffer is too slow for real training (pure Python). FastPERBuffer uses numpy arrays and returns torch tensors for GPU training. AuditedPERBuffer wraps it with a shadow exact buffer for correctness spot-checking.

3. How to Run the Project

Setup

```
brew install uv
cd /Users/joshuawu/Reservoir
uv sync
```

Tests (all 168)

```
uv run pytest tests/ -v
# or
make check
```

Campaigns

```
uv run python -m campaigns.mutation      # T4: forgery rejection (63/63)
uv run python -m campaigns.crash          # T3: crash atomicity (70/70)
uv run python -m campaigns.divergence     # T2: float comparison
```

Reservoir - Implementation Plan

End-to-End Demo

```
uv run python -m demo.tiny_dqn
# Expected:
#   Run A: Total reward: 25.0
#   Run B: Total reward: 25.0
#   Q(right) > Q(left) all states 0-3
#   Attestation VERIFIED
#   Bitwise-identical: YES
```

TLA+ Model Check

```
bash spec/check.sh
# No errors found, 44,611 states explored
```

4. Implementation Plan

Risks

Level	Risk
HIGH	T2 full campaign (540K workloads) is slow in pure Python - needs FastPERBuffer-backed exact tree or Rust/C extension
MEDIUM	PDF read failed (missing poppler) - install brew install poppler to verify paper matches code
MEDIUM	Application form (web app vs CLI vs library) is undefined - needs direction
LOW	TLA+ requires Java + TLA+ Toolbox installed separately

Phase 0: Environment & Paper Access

- [] brew install poppler (to read reservoir.pdf)
- [] Confirm: uv run pytest tests/ -v passes all 168 tests
- [] Confirm: uv run python -m demo.tiny_dqn produces expected output

Phase 1: Full Benchmark Reproduction

- [] Run T3: uv run python -m campaigns.crash
- [] Run T4: uv run python -m campaigns.mutation
- [] Run T2 reduced: uv run python -m campaigns.divergence
- [] Assess feasibility of full 540K-workload T2 grid (may need C/Rust accelerator)

Phase 2: Application - Choose Direction

Three natural application directions (pick one or combine):

Option A - PyPI Library (most impactful for RL community)

- Clean public API: from reservoir import FastPERBuffer, ExactPERBuffer
- Proper pyproject.toml metadata and entry points
- Published to PyPI via uv publish

Option B - Benchmark / Audit CLI

- reservoir-bench command: runs any campaign with configurable parameters
- reservoir-audit command: verify an attestation file against a seed
- Rich terminal output with tables and progress bars

Option C - Web Dashboard

- FastAPI backend exposing buffer state, sampling distributions, attestation records
- React/Plotly frontend showing priority distribution, sampling heatmaps, audit trail
- Useful for teaching and debugging PER behavior

Phase 3: Benchmarks vs Paper

- [] Port full T2 grid (3 caps x 3 magnitudes x 3 ratios x 10,000 workloads) against FastPERBuffer

Reservoir - Implementation Plan

- [] Benchmark FastPERBuffer throughput vs SB3 baseline (sketched in README)
- [] Run Gymnasium training comparison: ExactPERBuffer + AuditedPERBuffer vs standard SB3

5. Performance Reference (FastPERBuffer vs SB3)

Metric	reservoir FastPERBuffer	SB3 ReplayBuffer (uniform)
Insert	5 us	2 us
Sample (batch=256, cap=100K)	0.12 ms	0.03 ms
IS weights	Yes	No
Priority updates	Yes	No
GPU tensors	Yes	Yes
Correctness audit	Yes (unique)	No

The remaining gap vs SB3 uniform is inherent: PER requires $O(\log N)$ tree traversal per sample, while uniform sampling is $O(1)$.